

IMPLEMENTATION PLAN

Enterprise Data Compiler

MVP Implementation Plan

Scope	Revenue-to-Cash Evidence Loop; first production cohort
Authoring	Codex and Claude desktop clients over MCP
Reference flow	Salesforce + PostgreSQL → Snowflake → dbt → governed context
Status	Pre-implementation funding and execution baseline
Version	1.4
Date	August 2026

Goal

Prove one managed, deterministic Revenue-to-Cash outcome: authoritative commercial and operational data, a version-bound finance product, reproducible lifecycle exceptions, and governed agent context—without customer-operated clusters, brokers, or pipeline runtimes.

This plan defines architecture, workstreams, gates, and evidence. It intentionally omits implementation APIs, wire formats, and deployment manifests.

Contents

- 1 Executive Decision.....3
- 2 Scope and Non-goals.....4
- 3 System Architecture.....7
- 4 Desktop Authoring over MCP.....8
- 5 Elastic Execution Model.....10
- 6 End-to-End Data Reliability.....11
- 7 Capability Provider Kit.....14
- 8 API and OAuth Connectivity.....15
- 9 Kafka Decision.....17
- 10 Managed Operations.....17
- 11 Security and Tenant Isolation.....19
- 12 Implementation Workstreams.....20
- 13 Phased Delivery Plan.....21
- 14 Verification Strategy.....23
- 15 Production Cohort and Exit Gates.....24
- 16 Risks and Containment.....26
- 17 Immediate Next Actions.....27
- References.....29

1 Executive Decision

Recommendation: Fund a Revenue-to-Cash Evidence Loop for one production cohort. In one conversationally authored contract bundle, Salesforce commercial records and PostgreSQL order, subscription, invoice, and payment records are delivered to Snowflake; existing dbt models produce the finance data product; deterministic reconciliation detects lifecycle gaps; and governed MCP resources expose results to finance and sales agents. The platform does not implement a pipeline editor, another transformation framework, a general Spark service, or mandatory Kafka.

1.1 Objective

The MVP proves one complete enterprise outcome rather than a connector checklist: a closed-won Salesforce opportunity can be traced through operational order, payment, and subscription state into a governed finance-ready data product, with explicit freshness, PII, retention, lineage, and exception obligations. Users author and operate the bundle from Codex- or Claude-class desktop clients without placing credentials in chat. The compiler remains authoritative for meaning and legality; desktop clients remain replaceable source-language hosts.

Table 1. MVP decision summary

Dimension	Committed MVP choice	Deferred choice
Authoring	Host-neutral MCP server used from Codex and Claude desktop clients	Proprietary visual pipeline editor
Sources	PostgreSQL snapshot plus logical CDC; Salesforce bulk plus cursor acquisition	Additional databases, SaaS systems, and arbitrary REST
Destination	Snowflake staged load and key-based merge	Multiple warehouses and operational sinks
Transformation	Existing dbt project, manifest, tests, lineage, and model versions	Compiler-owned join or SQL framework
Business proof	Opportunity→order and payment→subscription reconciliation with evidence	General business-process engine
Consumption	Governed finance/sales MCP resources with PII and freshness policy	Unrestricted SQL or agent write authority
Execution	Leased immutable partition work units on managed elastic workers	General Spark or user-managed cluster
Reliability	Pinned input boundary, idempotent commit, checkpoint-after-receipt, reconciliation	Universal exactly-once claim
Messaging	Managed work queue, durable state store, object staging, append-only evidence	Customer-operated or mandatory Kafka
Networking	Managed public access plus outbound-only private Relay	Customer runtime farms

1.2 Success definition

The first cohort is successful when the complete Revenue-to-Cash bundle operates for 30 consecutive days: source contracts meet their envelopes, the dbt data product is version-bound and tested, declared lifecycle exceptions are detected within their windows, governed agent answers retain freshness and

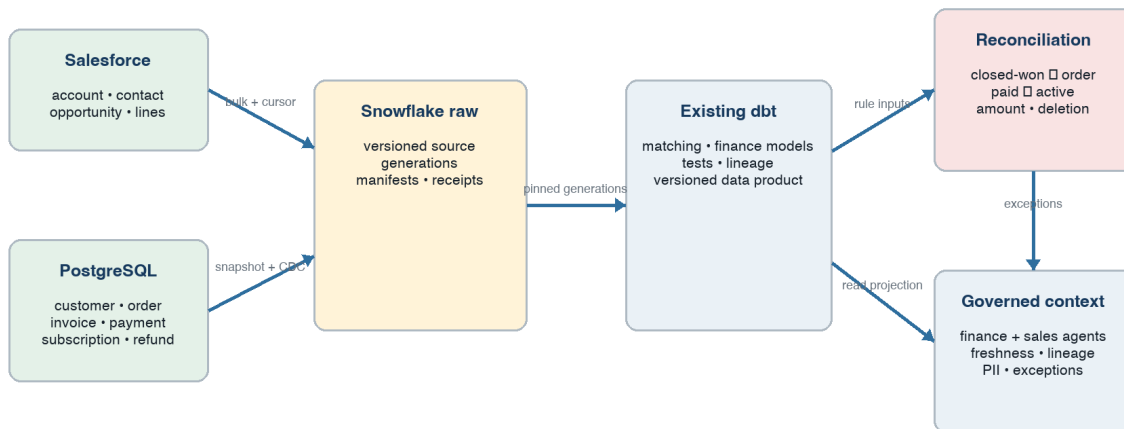
provenance, injected failures recover without unexplained divergence, and every material authoring and runtime decision is replayable. The cohort must also reproduce one L01 snapshot-mirror→incremental-mirror admission with instantiated D1–D8 obligations and one rule-table-miss rejection with No Valid Plan, an absent-rule explanation, unmet obligations, and a smallest-change counterfactual. Solver-derived minimal unsatisfiable cores remain post-MVP under the architecture specification's constraint-solving work. This validates the curated legality mechanism; it does not validate general equivalence, cost-based optimization, autonomous migration, or optimizer stability. Salesforce write-back remains a separately approved post-MVP gate.

2 Scope and Non-goals

2.1 Coherent business story

A B2B subscription business manages accounts, contacts, products, and opportunities in Salesforce. Its application records customers, orders, subscriptions, invoices, payments, refunds, and cancellations in PostgreSQL. Finance uses Snowflake and an existing dbt project for bookings, billings, collections, subscription state, and net revenue. The platform continuously proves whether closed business becomes an order and whether paid orders become active subscriptions, then exposes the governed result and exceptions to finance and sales agents.

One coherent outcome: Revenue-to-Cash Evidence Loop



The compiler binds contracts and evidence; dbt retains transformation ownership; agents receive read-only governed projections.

Figure 1. Revenue-to-Cash Evidence Loop and ownership boundaries.

2.2 Competitor-pattern synthesis

The story combines use-case patterns that incumbent products normally demonstrate separately. The MVP is not a compatibility clone. Each pattern is admitted only where it tests a required architectural boundary.

Table 2. Patterns synthesized into one outcome

Observed product pattern	MVP use	Architectural consequence
Fivetran-style managed replication	Salesforce and PostgreSQL arrive in Snowflake on a declared schedule without customer runtime administration	The service owns freshness, retries, connector release, schema response, and incident detection
Airbyte-style connector extensibility	The three reference capabilities are built through the same provider kit available to future teams	Connector speed is measured without allowing provider-local schedulers, secrets, or retry semantics
SnapLogic-style low-friction authoring	A concise conversational flow discovers assets, resolves ambiguity, previews effects, and activates a verified digest	Interaction is replaceable; semantic authority remains server-side
Informatica-style governed customer/revenue view	An existing dbt project binds commercial and operational records into a finance-owned data product	The compiler consumes catalog, ownership, policy, lineage, and tests; it does not create another catalog or transformation system
MuleSoft-style operational API access	The governed outcome is exposed to finance and sales through read-only MCP resources	Access is purpose-bound, policy-filtered, and evidence-bearing rather than an unrestricted action bus

Table 3. Contract bundle

Contract	Required behavior	Mechanism proven
A. Salesforce commercial acquisition	Account, Contact, Opportunity, lines, product, pricebook, owner; OAuth; bulk snapshot; cursor updates; declared deletes	SaaS discovery, OAuth lifecycle, pagination, quotas, schema evolution
B. PostgreSQL revenue operations	Customer, Order, lines, Subscription, Invoice, Payment, Refund; consistent snapshot; logical CDC; private Relay option	Snapshot/CDC handoff, deletes, replay, fencing, private reachability
C. Snowflake raw generations	Isolated stage, manifest validation, idempotent merge, versioned schema and commit receipt	Destination ambiguity resolution and checkpoint-after-receipt
D. Finance-ready data product	Invoke existing dbt models and consume manifest, tests, lineage, owners, and model versions	External transformation ownership; reproducible multi-source model binding
E. Revenue reconciliation	Closed-won→order, paid order→active subscription, amount, invoice/payment, deletion and cancellation invariants	Deterministic cross-system evidence and incident generation
F. Governed agent context	Finance/sales summaries, lifecycle status, exceptions, freshness, lineage; PII and purpose controls	Read-only MCP resources with policy and provenance

2.3 Permitted semantics

- Source execution contracts remain single-source; the compiler does not implement the cross-source join.
- Existing dbt models own Salesforce-to-operational customer matching and finance transformations.
- The compiler binds the dbt project, manifest, tests, lineage, source generations, owners, and model version into the data-product contract.

- Reconciliation evaluates only the contract's declared reconciliation scope and named exclusion classes; ambiguous mappings remain excluded and are not resolved by the compiler.
- Filter, projection, rename, compatible cast, mask, key selection, and key-based upsert remain the only compiler-native transforms.
- Freshness is measured source observation→Snowflake visibility and source visibility→data-product visibility.
- At-least-once acquisition uses idempotent destination application; deletion behavior is fixed in activation.
- T0 and T1 contracts only; recommendation or approval mode; no autonomous plan migration.

2.4 Explicit exclusions

- Compiler-native multi-source joins, aggregations, arbitrary code, or replacement for dbt.
- Payment initiation, subscription provisioning, email delivery, or general business-process orchestration.
- Salesforce mutation in the required MVP; bounded reconciliation-status write-back is post-MVP.
- Bidirectional synchronization and non-repeatable external actions.
- General Spark execution, user-controlled clusters, and customer Kafka administration.
- Streaming windows, event-time joins, broad event routing, or arbitrary REST workflows.
- Connector marketplace, more than three reference capabilities, or claims of universal API support.
- Cross-region active-active execution, autonomous stateful optimization, standalone authoring application, or editable execution graph.

3 System Architecture

Desktop-authored, compiler-controlled Revenue-to-Cash MVP

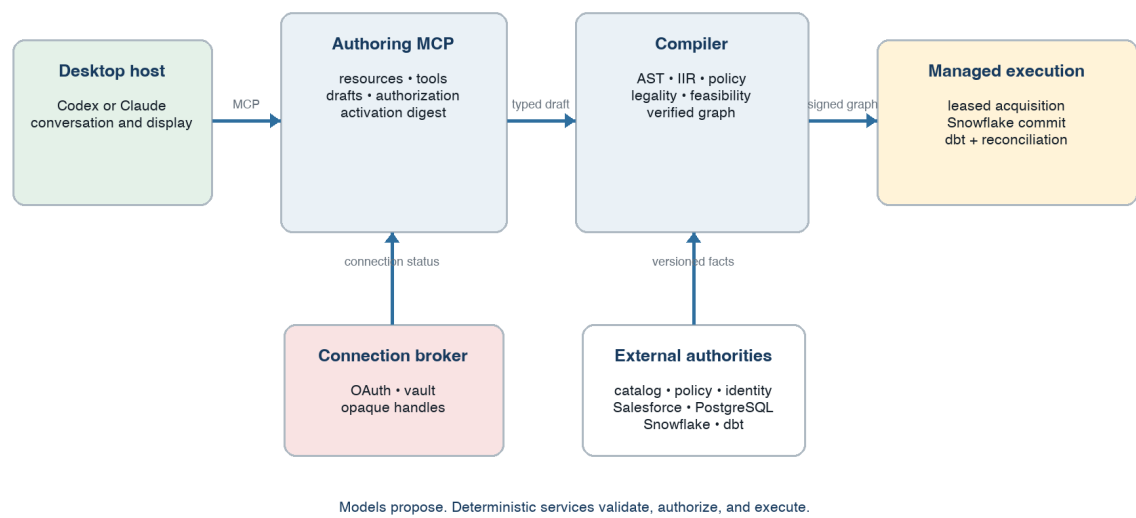


Figure 2. MVP trust boundary from desktop authoring through Revenue-to-Cash execution and exposure.

3.1 Control-plane components

Table 4. Control-plane responsibilities

Component	Owns	Must not own
Contract Authoring MCP Server	Host-neutral tools/resources, sessions, authorization filtering, draft mutations	Semantic validity or execution state
Contract Service	Versioned drafts, activation digests, approvals, lifecycle	Credentials or physical scheduling
Compiler	AST, semantic IIR, resolution, policy, legality, feasibility, deployment compilation	Runtime retries or record-level nondeterminism
Connection Broker	OAuth attempts, callbacks, token exchange/rotation, opaque connection handles	Raw secrets in MCP results
Provider Registry	Versioned declarations, conformance tier, evidence validity	Trust based on provider assertion alone
Coordinator	Partition epochs, leases, fencing, checkpoints, migration admission	Global execution serialization
Evidence Service	Append-only contract, plan, execution, reconciliation, and incident facts	Unverifiable health synthesis
dbt Adapter	Version-pinned invocation, manifest/test/lineage observation, model-run evidence	Business transformation semantics or SQL authoring
Reconciliation Evaluator	Declared lifecycle predicates, deadlines, exceptions, evidence links	Source mutation or probabilistic matching

Component	Owns	Must not own
Context Exposure	Authorization-filtered finance/sales MCP resources and freshness/provenance	Unrestricted SQL or action authority

3.2 Data-plane components

Managed runtime cells execute signed graphs. Stateless workers pull leased partition work, resolve short-lived grants, read a bounded input, produce immutable staged output, commit through a provider-defined idempotent protocol, and emit evidence. An outbound-only Relay performs the same restricted capability invocation for private PostgreSQL. It has no planning authority and owns state only when named by a contract partition and epoch.

3.3 Durable stores

Table 5. Durable store boundaries

Store	Contents	Consistency requirement
Contract store	Drafts, versions, approvals, activation digests	Serializable lifecycle transitions per contract
Coordination store	Epochs, leases, fences, checkpoints, pending commits	Conditional writes and monotonic epochs
Object staging	Immutable extraction segments, manifests, replay inputs	Content integrity and retention bounds
Evidence log	Decision, runtime, reconciliation, and incident events	Append-only, tamper-evident ordering per contract
Secret store	OAuth tokens, keys, database credentials	Envelope encryption, scoped retrieval, rotation

3.4 Cross-source semantics

The source contracts create immutable, versioned Salesforce and PostgreSQL generations. An external dbt run consumes those exact generations and emits a versioned finance data product. The compiler records the dbt project digest, manifest, selected model graph, test results, source-generation identities, and destination relation generation. Reconciliation evaluates declared predicates over the materialized data product and linked source evidence. If matching keys or dbt tests are Unknown, the data product cannot become Conforming.

4 Desktop Authoring over MCP

4.1 Client trust model

Codex and Claude are untrusted conversational frontends. They may propose structured intent, call authorized tools, and explain deterministic results. They cannot validate semantics, relax policy, handle

raw credentials, produce an execution graph, or turn natural-language assent into activation authority. All durable state is server-side; conversation history is context, not the system of record.

Table 6. MCP surface

Surface	MVP operations	Effect class
Resources	Draft, activation summary, revenue product, lifecycle status, exceptions, freshness, lineage, evidence, incident timeline	Read-only and authorization-filtered
Discover	Find authorized datasets, endpoints, objects, and concepts	Read-only
Draft	Create draft; apply intent delta; record clarification	Reversible control-plane mutation
Analyze	Resolve, validate, solve, and explain feasibility	Read-only compilation
Connect	Begin OAuth or secure credential-entry flow	Security-sensitive control operation
Verify	Create immutable activation summary and verification digest	Read-only compilation with retained evidence
Activate	Activate an exact verified digest	Production data effect; explicit approval
Operate	Pause, resume, reauthorize, request isolated backfill	Operational effect

4.2 Authoring state machine

1. The client discovers only authorized Salesforce, PostgreSQL, Snowflake, dbt, catalog, and policy metadata.
2. The user states the Revenue-to-Cash outcome; the client submits a bounded contract-bundle proposal.
3. The compiler resolves source objects, destination, dbt models, matching-key authorities, PII, retention, freshness, consumers, and reconciliation predicates.
4. The client asks only material questions, including authoritative customer key, deletion behavior, exception windows, and permitted agent roles.
5. Verification emits one activation summary with source contracts, dbt binding, reconciliation invariants, exposure, defaults, cost, and evidence requirements.
6. The client presents that exact digest and invokes activation only after explicit user action.
7. The server rechecks identity, roles, digest, evidence age, policy, provider attestations, and dbt compatibility before creating source epochs and the initial data-product generation.

4.3 Activation invariant

Hard boundary: If any contract field changes after verification, the activation digest changes and prior approval is invalid. The server never accepts client prose such as “the user approved” as authorization.

4.4 Host portability

The compatibility baseline uses MCP tools, resources, structured results, Markdown summaries, browser links for OAuth, and polling or notifications for long operations. Correctness cannot depend on client-specific widgets, hidden prompts, local files, conversation replay, or one vendor's confirmation UI. The same Revenue-to-Cash draft, activation summary, exception investigation, and operational context must be resumable from another conforming desktop host.

5 Elastic Execution Model

5.1 RDD lesson without Spark coupling

The useful RDD properties are immutable partition descriptions, deterministic functions, lineage, data-local scheduling where possible, and recomputation after worker loss. The MVP adopts those properties as compiler-generated partition work units. It does not embed Spark as a customer-visible runtime. Mutable external sources require stronger input identity than RDD lineage alone: a retry must name a snapshot, log position, stable API cursor interval, or persisted extraction segment.

Table 7. *Partition work-unit contract*

Field	Purpose
Contract and graph digest	Pins semantic and physical authority
Partition identity	Stable unit for lease, evidence, and retry
Bounded input	Snapshot range, LSN interval, object/time window, or page cursor
Transform fragment	Deterministic IIR-derived operations
Output generation and batch identity	Isolates staging and resolves retries
Checkpoint rule	Defines the evidence required before progress advances
Resource envelope	Memory, CPU, bytes, duration, and provider quota

5.2 Partitioning

Table 8. *MVP partition strategies*

Capability	Initial strategy	Skew and limit response
PostgreSQL snapshot	Primary-key ranges under one consistent snapshot	Split ranges using sampled quantiles; isolate unsplittable hot ranges
PostgreSQL CDC	LSN-bounded stream partitioned only where ordering permits	Backpressure at source slot; do not fabricate parallel order
Salesforce snapshot	Object plus provider-supported bulk query batches	Respect job and daily API quotas
Salesforce incremental	Object plus non-overlapping cursor/time windows	Overlap and deduplicate when cursor semantics require it
Snowflake load	Immutable stage files grouped by destination batch	Resize file groups within destination concurrency envelope

5.3 Scaling controller

The controller computes desired concurrency from backlog bytes and records, observed per-partition throughput, freshness slack, source and destination quotas, tenant envelope, worker availability, and skew confidence. Scale-out is capped by the minimum provider and tenant limit. Scale-in drains leases at checkpoint boundaries. Missing or stale rate evidence produces conservative admission. Users never select executors, partitions, worker memory, or cluster size.

First-cohort target: Treat the source-load ceiling as a declared contract parameter. Demonstrate elastic scale-out and scale-in against each partner only within its DBA-approved ceiling, while preserving destination correctness and freshness accounting. Separately demonstrate 1→100 concurrent snapshot partitions against a synthetic or isolated source with the same work-unit and commit protocol.

6 End-to-End Data Reliability

Checkpoint advances only after durable destination evidence

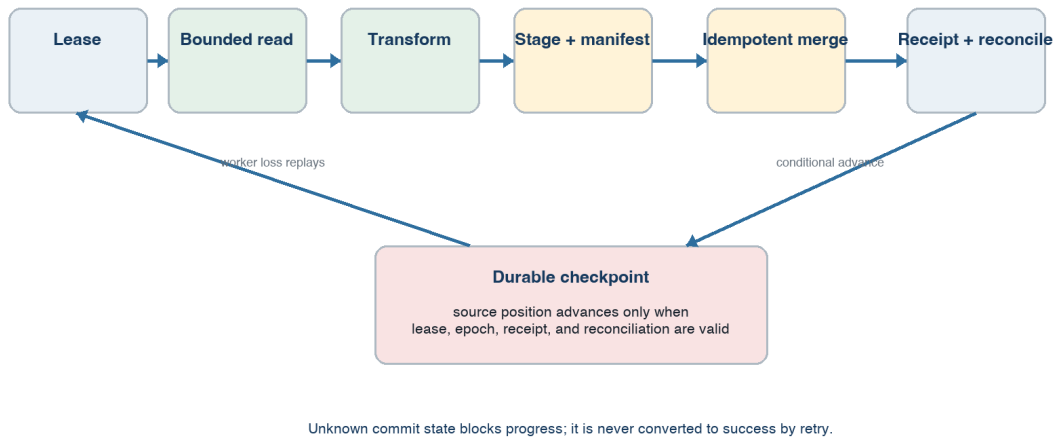


Figure 3. Partition processing and checkpoint-after-commit protocol.

6.1 Reliability claim

The MVP guarantees that destination histories stay within the activated contract when provider prerequisites hold. It does not claim each internal action executes exactly once. Acquisition may repeat; destination application is idempotent; checkpoints advance only after durable commit evidence. Unknown commit state blocks progress until resolved.

6.2 Partition commit protocol

1. Acquire a lease bound to contract, partition, and epoch.

2. Read the declared bounded input and compute a content manifest.
3. Apply deterministic transformations and write immutable stage objects under a stable batch identity.
4. Validate stage count, schema, keys, checksums, and deletion manifest.
5. Invoke an idempotent Snowflake merge or generation switch using the batch identity.
6. Persist the destination receipt and reconciliation result.
7. Conditionally advance the checkpoint if the lease and epoch remain current.
8. Emit completion evidence and release the lease.

6.3 Snapshot-to-CDC handoff

1. Establish a consistent PostgreSQL snapshot and corresponding retained-log position.
2. Read snapshot ranges in parallel while retaining changes after the cut position.
3. Commit and reconcile the snapshot generation.
4. Replay retained changes from the exact cut position.
5. Enter steady-state CDC only when the compiler/runtime proves there is no uncovered interval.

6.4 Failure responses

Table 9. Failure and required response

Failure	Required response	Forbidden response
Worker loss	Expire lease; replay from last durable checkpoint	Advance progress from worker memory
Duplicate input/page	Deduplicate by source identity and batch manifest	Assume provider pagination is unique
Ambiguous Snowflake commit	Query by batch identity; resolve committed/not committed/Unknown	Blindly repeat and report success
Schema incompatibility	Pause affected partition; recompile and require approval when material	Implicit coercion
Source throttle	Backpressure; honor reset evidence; preserve freshness violation	Unbounded retry concurrency
Checkpoint-store partition	Finish only bounded work; do not commit without valid fence	Create a second owner
Reconciliation mismatch	Quarantine generation and retain prior visible state	Publish and repair later silently

6.5 Reconciliation

Every snapshot partition records source count, manifest count, unique-key count, destination applied count, deletion count, and content hash where the provider can supply stable values. CDC records source positions, event identities, applied counts, duplicate intervals, and watermark. Sampling can supplement but never replace exact key and commit checks required by the contract.

Reconciliation evaluates only records within the contract's declared reconciliation scope (§15.1). The scope, excluded reason classes, and observed counts are versioned contract fields. Every result reports eligible, in-scope, and excluded counts by class, plus resulting coverage.

6.6 Revenue lifecycle invariants

Table 10. Revenue-to-Cash reconciliation

Invariant	Deterministic evaluation	Failure disposition
Scope admission	Apply lifecycle predicates only to records admitted by the declared reconciliation scope	Report exclusions by named class; never classify them as lifecycle violations
Closed-won opportunity produces an order	Join on the contract-authorized key; require order by the partner-specific business-transition deadline	Open within detection deadline; route to contract-named sales-operations owner
Paid order produces an active subscription	Match settled payment and order key to active subscription by partner-specific business-transition deadline	Open within detection deadline; route to contract-named finance/revenue-operations owner
Opportunity and order amounts agree	Compare normalized currency and declared tolerance using version-pinned dbt output	Open amount exception; do not rewrite source values
Invoice and payment reconcile	Invoice gross equals captured minus refund under declared status and currency rules	Open finance exception with model and source lineage
Deletion/cancellation propagates	Evaluate the activated source-specific delete or terminal-state rule	Block Conforming if required evidence or propagation is absent
Every exception is reproducible	Bind source generations, dbt digest, model output, rule version, evaluation time, and deadline	Unknown evidence yields Unknown, never a cleared exception

6.7 Deadline calibration

Before activation, a read-only shadow period measures eligible opportunity→order and payment→subscription latency distributions over at least one representative business cycle. Finance and sales owners approve each business-transition deadline using the observed distribution, contractual process obligation, sample size, exclusions, and confidence; the platform-detection deadline is specified separately. The compiler records both clocks and the baseline digest. It never mechanically converts an observed percentile into an obligation or treats a slow business process as platform lateness.

Each exception class also declares an accountable business owner, acknowledgement SLA, disposition SLA, allowed dispositions, and escalation path. The shadow baseline supplies the expected steady-state opening rate and confidence interval. Undispositioned items remain open, escalate at SLA expiry, and continue to count against cohort usefulness; they are never auto-cleared by age or a later successful batch.

Partner acknowledgement and disposition measure business-process capacity, not platform conformance. Undispositioned exceptions remain visible, open, and escalating, but do not count as platform failures.

7 Capability Provider Kit

7.1 Provider boundary

PostgreSQL, Salesforce, and Snowflake must be implemented through the same provider kit that future teams use. The dbt adapter uses a separate external-compute capability because it observes and invokes an existing transformation authority rather than acquiring or writing source data. No provider or adapter creates a private scheduler, retry loop, secret store, or state machine. The platform supplies leases, checkpoints, grants, evidence, backpressure, and fault injection.

Table 11. *Provider modules*

Module	Responsibility	MVP evidence
Discovery	Endpoint test, namespace/object listing, schema observation	Permission-filtered fixtures and pagination tests
Authentication adapter	Required auth modes and scope validation	Expiry, revocation, rotation, and denied-scope tests
Snapshot reader	Bounded, restartable extraction	Repeatability, range completeness, checksum fixtures
Incremental reader	CDC or cursor position and delete behavior	Restart, overlap, late event, and retention tests
Writer	Stage, validate, idempotent merge, commit resolution	Duplicate batch and ambiguous-commit tests
Error classifier	Retryable, throttled, auth, permanent, ambiguous	Mutation tests for every classification branch
Capability declaration	Semantics, limits, placement, evidence validity	Conformance tier and version-pair regression

7.2 Developer experience

- A local harness runs providers against deterministic source and destination fixtures.
- A recorder captures sanitized provider interactions and creates replay fixtures without secrets.
- A fault proxy injects timeouts, truncation, duplicate pages, throttling, token expiry, and ambiguous responses.
- The provider owns no deployment manifest for customer clusters; certified artifacts are signed and released through the managed runtime.
- The conformance report identifies passed claims, Unknown semantics, allowed risk tier, and planner eligibility.

7.3 Delivery targets

A provider for an already understood REST acquisition family should reach first local records within five engineering days and Provisional status within one to two provider-engineer months. Certified status for a materially new semantic family is planned at four to eight provider-engineer months plus independent review. Salesforce is expected at the upper end because certification must cover Bulk API 2.0 query partitioning where supported, formula and compound fields, field-level security, `queryAll`/`IsDeleted` behavior and retention, cursor-field semantics, quotas, and sandbox-to-production metadata drift. Its measured effort is the primary test of the conformance-cost assumption.

7.4 dbt integration boundary

The partner owns the version-controlled dbt project, execution environment, credentials, finance semantics, and named model owner. The platform holds narrowly scoped authority to trigger a dedicated pilot job in that environment; it does not rely on the partner's general scheduler to satisfy a product generation. Each invocation pins the project revision, selected model graph, target namespace, and generation-specific Snowflake source relations. The adapter compares the relations and generation identities actually consumed, manifest, run results, required tests, and produced namespace with the pinned set. A concurrent partner run publishes elsewhere and cannot satisfy the receipt. Any mismatch makes the candidate generation Unknown while the last Conforming generation remains visible. The compiler may explain or propose a change, but never generates, approves, or silently modifies finance SQL.

8 API and OAuth Connectivity

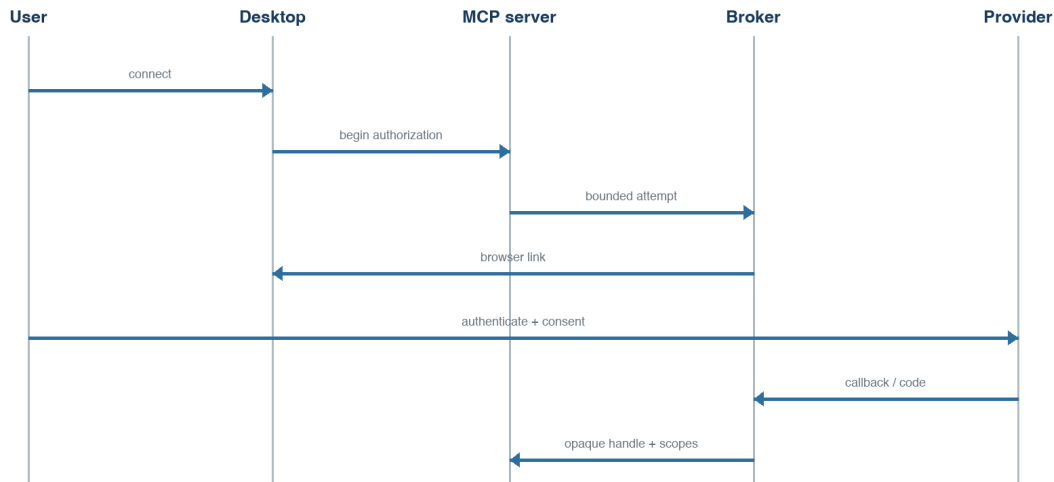
8.1 Authentication coverage

Table 12. MVP authentication modes

Mode	MVP use	Required controls
OAuth authorization code plus PKCE	Salesforce delegated connection	State/nonce validation, bounded attempt, exact redirect, scope display
OAuth refresh rotation	Long-running SaaS acquisition	Atomic token replacement, replay detection, expiry alert
OAuth client credentials	Service-owned APIs where supported	Audience and scope pinning; no user impersonation
Database secret	PostgreSQL	Secure entry, encryption, rotation, least-privilege test
Key pair	Snowflake	Encrypted private key, fingerprint display, rotation overlap
API key	Provisional future REST provider	Header/query placement declared; redaction and rotation

8.2 OAuth sequence

OAuth tokens remain outside the desktop model



No token, secret, or private key enters MCP resources, tool results, contracts, or model context.

Figure 4. OAuth authorization remains outside the desktop model and MCP transcript.

The MCP Connect tool creates a bounded authorization attempt and returns a platform browser link. The broker validates the callback, exchanges and seals tokens, verifies granted scopes, and returns only an opaque connection handle plus sanitized capability status. Neither desktop host nor model sees access tokens, refresh tokens, client secrets, database passwords, or private keys.

8.3 Generic API acquisition substrate

The MVP implements reusable HTTP transport for DNS/TLS validation, proxy policy, connection pooling, timeouts, pagination primitives, quota accounting, `Retry-After`, bounded retries, content-size limits, schema decoding, and redaction. It is an internal provider substrate, not an end-user arbitrary HTTP connector. A new API becomes usable only after its provider declares cursor, deletion, ordering, retry, and authentication semantics and passes the corresponding suite.

8.4 Post-MVP Salesforce write-back

The first operational action is deliberately narrow: set a reconciliation-status field and verified order identifier on an Opportunity. It requires a stable action identity, expected prior record version, field-level authorization, contract-owner and policy-authority approval, idempotent update semantics, read-after-write verification, and separate action evidence. It cannot change stage, send email, initiate payment, provision service, or retry an ambiguous write without positive resolution. This slice begins only after the 30-day read/reconciliation gate.

9 Kafka Decision

9.1 Decision

Not in the required path: Do not make Kafka a mandatory internal record bus for the MVP. Use a managed work queue for leases, a coordination store for epochs and checkpoints, object storage for immutable stage/replay segments, and an append-only evidence log. Kafka adds a commit boundary without resolving external-system correctness.

Table 13. Kafka comparison

Concern	Kafka provides	MVP still requires
Partition progress	Consumer offsets	Contract epoch, source position, sink receipt, and conditional checkpoint
Ordering	Per-partition order	Provider and contract ordering scope across acquisition and commit
Replay	Retained log replay	Pinned mutable-source boundary and destination idempotency
Exactly-once	Transactional semantics within supported Kafka flows	Resolution across PostgreSQL/Salesforce and Snowflake boundaries
Fan-out	Multiple consumer groups	Not required for the single Revenue-to-Cash data product and its read-only projections
Evidence throughput	Ordered append, retention, and scalable consumption	Tamper-evident contract ordering, policy retention, searchable decisions, and commit-path backpressure
Operations	Mature ecosystem	A managed service or another distributed system to own

9.2 Revisit trigger

Reconsider Kafka or another durable log when an admitted contract requires independent consumer fan-out, long replay retention, event routing, streaming-window semantics, or evidence write throughput and retention that the evidence service cannot satisfy economically. At that point Kafka is a Capability Provider with pinned topics, partitions, offsets, retention, schema, and consumer-group semantics; it is not assumed to make arbitrary endpoints exactly once.

10 Managed Operations

10.1 Service ownership

The platform team owns whether active contracts run on time. Customers authorize access, approve semantics, and respond to business-policy decisions; they do not size clusters, restart workers, tune partitions, maintain brokers, rotate internal state, or reconcile silent divergence.

Table 14. Operational control loops

Loop	Inputs	Automatic action	Escalation condition
Freshness	Source observation, destination receipt, backlog, quotas	Scale within envelope; reprioritize; backpressure	No feasible capacity before deadline
Authentication	Token expiry, refresh result, scope drift	Refresh or rotate	Consent revoked, invalid grant, scope loss
Schema	Observed delta and dependency graph	Continue only with neutrality proof	Bound field, key, policy, or value semantics change
Provider health	Error classes, throttle evidence, conformance contradiction	Circuit break and bounded retry	Semantic contradiction or sustained outage
Data product	Pinned source generations, dbt run state, required tests, publication deadline	Retry eligible external run; retain last conforming generation	Required test failure, incompatible manifest, or deadline infeasible
Revenue reconciliation	Lifecycle rules, evidence times, exception deadlines	Re-evaluate on new conforming generation; correlate related failures	Deadline expires, evidence is Unknown, or exception volume breaches envelope
Cost/capacity	Measured rate and spend envelope	Recommend alternative plan	Budget or provider ceiling infeasible

10.2 Contract states visible to users

- **Healthy** — source, data-product, reconciliation, and exposure obligations are measured and satisfied.
- **At risk** — evidence, capacity, quota, credential, or dbt run approaches a bound.
- **Late** — a source or data-product freshness objective is currently violated.
- **Blocked** — safe execution cannot continue, including authentication or required-test failure.
- **Indeterminate** — an external commit may or may not have completed.
- **Quarantined** — integrity or security evidence prevents publication.
- **Paused** — authorized user or operator has stopped acquisition at a checkpoint.

10.3 On-call and service review

Every incident is keyed by bundle, contract, plan, epoch, partition, data-product generation, rule version, and provider fault domain. The on-call view shows last safe checkpoints, affected finance generation, freshness debt, failed dbt tests, revenue exceptions, ambiguous commits, valid mitigation choices, and customer-visible consequence. Weekly review covers source and product freshness, reconciliation deadlines, mean recovery time, provider regressions, OAuth failures, false or missed exceptions, context-access denials, and manual interventions.

11 Security and Tenant Isolation

11.1 Required controls

- Tenant identity is bound at every MCP request, contract lookup, compiler snapshot, graph, lease, grant, state key, and evidence event.
- Authorization-filtered discovery cannot reveal names or counts of inaccessible assets.
- Secrets are stored only in the broker/vault and resolved through scoped short-lived grants.
- Execution graphs and Relay fragments are signed, immutable, version-pinned, and bounded by tenant, capability, endpoint, operation, and time.
- Managed runtime cells enforce tenant resource envelopes and separate staging namespaces, state prefixes, grants, and encryption context.
- Provider code runs with the minimum capability grant and no control-plane database access.
- MCP mutations, OAuth events, approvals, releases, and operational overrides are retained as audit evidence.

11.2 Relay boundary

The Relay initiates outbound control connectivity, validates signed fragments, resolves local credentials when configured, invokes only declared private capabilities, and buffers bounded evidence. It exposes no public listener for normal operation, performs no planning, accepts no arbitrary shell or code execution, and cannot extend its own lease or grant.

11.3 Threat-driven tests

Table 15. Threat-driven acceptance tests

Threat	Required negative test	Expected result
Cross-tenant reference	Substitute another tenant's contract, connection, provider, state, and evidence identifiers	Uniform authorization failure with no existence disclosure
Prompt-injected tool call	Untrusted metadata requests activation or broader discovery	Typed proposal is rejected or requires the normal approval path
Grant replay	Reuse an expired or wrong-endpoint capability grant	Runtime rejects before provider access
Stale graph	Run a graph after contract or provider version invalidation	Admission fails or bounded last-valid policy applies
Relay compromise	Attempt arbitrary destination, operation, secret, or unsigned fragment	Relay rejects; blast radius remains within the issued grant
Secret exfiltration	Force provider errors and MCP explanations containing credentials	Redaction blocks release and creates a security event

12 Implementation Workstreams

12.1 Decomposition

Table 16. *Workstream ownership*

Workstream	Primary deliverable	Depends on
A. Semantic kernel	Contract-bundle lifecycle, AST/IIR, validation, legality, feasibility, activation digest	Enterprise metadata adapters
B. Desktop authoring	MCP server, Revenue-to-Cash conversation, Codex and Claude compatibility suite	A, identity, connection broker
C. Connection security	OAuth/secret broker, scope validation, rotation, opaque handles	Identity and vault
D. Provider kit	Declarations, local harness, recorder, fault proxy, conformance reports	A and C
E. Reference providers	Salesforce acquisition, PostgreSQL snapshot/CDC, Snowflake commit	D
F. Runtime and state	Work units, leases, epochs, checkpoints, scaling, Relay	A, C, D, E
G. Data-product binding	dbt project/model binding, run observation, test and lineage evidence, publication generations	A, E, F
H. Reconciliation and context	Lifecycle evaluator, exception projections, governed finance/sales MCP resources	B, G, policy
I. Managed operations	Bundle health, control loops, incident resources, release and rollback	B, F, G, H
J. Security and assurance	Threat model, isolation, signing, audit, recovery and write-back gates	All workstreams

12.2 Team topology

Start M0 with 6–8 engineers and grow to 18–22 at peak: four semantic/control-plane and MCP, four runtime/state/Relay, four provider/connectivity, two data-product/reconciliation, two security/assurance, two SRE/operations, and one to two product-design/program roles. No critical workstream has only one implementer; named backups cover compiler legality, state ownership, Salesforce, dbt publication, and incident control loops. Independent reviewers gate provider semantics, dbt publication, tenant isolation, activation authority, and any later write-back. Each partner supplies a data engineer, fractional Salesforce administrator, dbt owner, security reviewer, and finance/sales process owners during relevant phases.

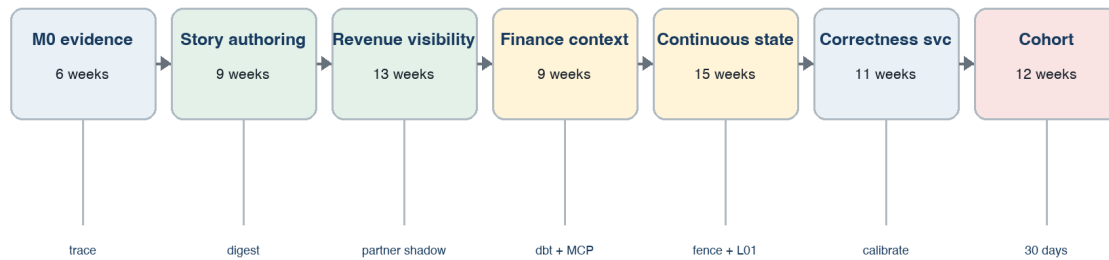
12.3 Repository boundary recommendation

- ``contract-compiler``: bundle lifecycle, AST/IIR, analysis, planner, verification.
- ``authoring-mcp``: resources/tools, host adapters, interaction conformance.
- ``connection-broker``: OAuth, secret entry, opaque handles, grants.
- ``provider-kit``: declarations, harness, recorder, fault injection, conformance.
- ``providers``: isolated PostgreSQL, Salesforce, and Snowflake packages.
- ``runtime``: graph supervisor, workers, leases, commit coordinator, Relay.

- `data-product-adapter`: dbt binding, invocation observation, manifest/test/lineage evidence.
- `reconciliation-context`: lifecycle rules, exceptions, governed read projections.
- `evidence-ops`: evidence ingestion, bundle health, incident projections.
- `system-assurance`: end-to-end story fixtures, fault campaigns, performance and isolation tests.

13 Phased Delivery Plan

Dependency-gated delivery: 18–21 month program envelope



Funding gates: evidence □ visibility □ finance context □ continuous correctness □ cohort

70–80 weeks with zero overlap; overlap is allowed only at named stable artifact boundaries.

Figure 5. Gated delivery sequence for the first production cohort.

Table 17. Delivery phases and gates

Phase	Duration	Deliverable	Exit gate
0. Evidence thin thread	6 weeks	One PostgreSQL order row to Snowflake; partner data/process inventory begins	Fresh row linked to contract, graph, input boundary, manifest, receipt, and evidence
1. Story authoring	8–10 weeks	Versioned six-contract bundle, dual-host baseline, partner-derived golden fixture	Same verified bundle resumes across hosts; stale approval and unauthorized discovery fail
2. Slice 1 — revenue visibility	12–14 weeks	Snapshots plus read-only partner metadata/process shadow; Salesforce security review already active	Fresh linked opportunity/order visible; identity and lifecycle baselines meet eligibility gates
3. Slice 1 — finance context	8–10 weeks	Pinned dbt data product, required tests/lineage, governed MCP resource	Finance agent reads a fresh revenue record with policy, source generations, model version, and timestamp
4. Slice 2 — continuous state	14–16 weeks	CDC/incrementals, deletes, epochs, fencing, backfills, two-week legality demonstration	No uncovered interval; L01 admission and rule-table-miss rejection are reproducible

Phase	Duration	Deliverable	Exit gate
5. Slice 2 — correctness service	10–12 weeks	Calibrated lifecycle clocks, managed control loops, private Relay	Every injected violation is detected; recovery preserves last conforming generation
6. Production cohort	12 weeks	Two partners run the complete story in coexistence; 30-day evidence window	All source, product, reconciliation, context, security, and operations gates are met

13.1 Schedule interpretation

The phase durations total 70–80 weeks with zero overlap. The committed envelope is 18–21 months, including partner security queues and contingency. Dependency-respecting overlap is limited: provider-kit and connection work may begin after M0 contract/grant interfaces stabilize; dbt binding may begin after Snowflake generation identity and receipt interfaces stabilize in Slice 1; reconciliation fixtures may begin after the data-product generation contract stabilizes; and cohort preparation may begin after continuous-state and security admission interfaces stabilize. No downstream gate is waived by overlap. The described overlap produces the optimistic end of the range; partner queues, hiring ramp, and contingency consume that gain, so the committed envelope holds even when overlap is not realized. Foundational Architecture Specification v0.3 retains an 18–24 month estimate for the broader compiler program, including cost intervals, shadow optimization, and migration research; this vertical plan exchanges some of that scope for Salesforce, OAuth, dbt, reconciliation, and context exposure. Funding remains gated at M0, revenue visibility, finance context, continuous state/legality, and cohort. Exclusions remain certification beyond three providers plus dbt adapter, go-to-market, and SOC 2/ISO work.

Partner dependency model

Partner-side work is scheduled, not assumed: evaluation agreement and data inventory in M0; Salesforce connected-app/security review from project start; customer-executed identity aggregates and process-latency observation in phases 0–2; dbt owner and dedicated-job approval before phase 3; DBA source-load ceiling and Relay review before phase 4; finance/sales rule approval before phase 5. Partner delay consumes schedule contingency and cannot be hidden as engineering overlap.

Hiring ramp assumption

Recruiting begins before M0 for compiler-legality and state-ownership leads; at project start for Salesforce/provider, security, and SRE roles needed by phases 1–2; and by phase 1 for dbt/reconciliation and cohort-operations capacity needed by phases 3–5. A new engineer is counted as productive only after an assumed eight-week onboarding period. Table 17 durations are ramp-adjusted within the 18–21 month envelope. Absence of the compiler-legality lead blocks phase 1, the state-ownership lead blocks phase 2, and the Salesforce-semantics lead blocks Salesforce certification. If a blocking role is not productive at its phase gate, that phase does not start; scope and funding are re-sequenced rather than silently stretching the duration.

13.2 Post-MVP Slice 3 — bounded operational action

Only after the cohort gate may the team evaluate Salesforce reconciliation-status write-back. Admission requires a separate action contract, optimistic concurrency, idempotency and ambiguity-resolution evidence, policy approval, audit retention, and a shadow period. It is not required to declare the MVP complete and cannot inherit approval from the read-only bundle.

14 Verification Strategy

14.1 Test layers

Table 18. *Verification layers*

Layer	Required evidence
Semantic	Golden six-contract bundles, ambiguity cases, policy conflicts, legality negatives, deterministic compilation replay
Provider	Fixture replay, property tests, mutation tests, fault proxy campaigns, version-pair regression
Runtime	Lease/fence model checks, worker kill, duplicate delivery, checkpoint loss, corrupt stage, ambiguous commit
Data product	Pinned dbt manifest and source generations, selected graph, required-test results, lineage, publication receipt
Reconciliation	Golden lifecycle cases, deadline boundaries, currency/tolerance rules, deletes, Unknown evidence, exception reproducibility
End-to-end	Fresh opportunity, order, payment, and subscription traced through Snowflake, dbt, reconciliation, and governed context
MCP	Host matrix, unauthorized discovery, malformed calls, stale digest, duplicate activation, reconnect/resume
Security	Cross-tenant substitution, purpose/PII denial, prompt-injected metadata, grant replay, signature failure, secret redaction
Scale	Partner ceiling tests plus synthetic 1→100 partitions, skew, quotas, control-plane outage, mass invalidation
Operations	Stuck source and dbt runs, missed exception deadline, alert correlation, restore, provider rollback, expired evidence

14.2 Reliability fault campaign

- Kill a worker before stage completion, after stage completion, during merge, and after merge before checkpoint.
- Duplicate and reorder Salesforce pages; expire OAuth during a bulk job; revoke consent during steady state.
- Throttle PostgreSQL, Salesforce, and Snowflake independently and together.
- Break the control plane while graphs, grants, and leases have different remaining validity.
- Inject a broad provider attestation invalidation and measure revalidation fan-out.
- Corrupt a staged object and a checkpoint replica; demonstrate quarantine and reconstruction.
- Interrupt Relay update before switch, after switch, and during rollback.

- Force schema changes to a bound field, unbound field, key, deletion marker, and classified field.
- During the synthetic 100-partition test, measure evidence events/s, bytes/s, append p95/p99, backlog recovery, retained bytes, and contract-timeline query latency.

14.3 No proxy evidence

Release rule: A green build, responsive endpoint, healthy worker, populated warehouse, or agent answer is not proof. Every release candidate must create a new lifecycle transaction and trace it through source evidence, Snowflake receipt, dbt publication, rule evaluation, and authorization-filtered context; existing rows are fixtures.

15 Production Cohort and Exit Gates

15.1 Cohort shape

Select two B2B subscription partners that operate Salesforce, PostgreSQL, Snowflake, and a version-controlled, CI-tested dbt project with a named owner who accepts pilot pinning. Their in-scope opportunity→order and payment→subscription transitions must be system-driven rather than human- or batch-gated, verified through a read-only shadow baseline before signature. Under an evaluation agreement, each partner executes the unchanged aggregate 12-month identity screen: the share of eligible matured closed-won opportunities resolving to exactly one order, the share of eligible settled payments resolving to exactly one subscription, the ambiguous one-to-many rate, and every exclusion reason are measured and retained. The prior 98%, 99%, and 0.5% reference points remain reported for comparison but are not estate-wide eligibility gates. The screen scopes the contract; it does not silently filter it.

The declared reconciliation scope contains only records resolved unambiguously through the partner's authoritative mapping. It names and counts excluded reason classes—multi-order opportunity, amendment, renewal against an existing subscription, partial fulfilment, and missing mapping—with an accountable owner and approval authority. Coverage and exclusions are reported beside every reconciliation result; no exclusion is silent. Initial eligibility requires at least 250 matured opportunity transitions and 500 settled-payment transitions per year, at least 60% of each eligible population in scope, and no unexplained month-to-month membership drift above five percentage points for two consecutive months. Every threshold is subject to M0 calibration, not a universal platform rule. A provable claim over a declared subset with visible exclusions is valid; requiring 98% clean identity across the estate would require the customer to have already solved the problem the product is intended to expose. Exclude prospects requiring compiler-native probabilistic matching, payment actions, or transformation replacement.

15.2 Quantitative gates

Table 19. Cohort exit metrics

Metric	Gate
Authoring	Median T0 draft-to-ready under 5 minutes excluding external login; no more than 2 material questions

Metric	Gate
Host portability	100% of test drafts resume from Codex to Claude and vice versa without semantic delta
Source freshness	Per contract and declared window, $\geq 99\%$ meet objective in cohort; 99.9% is GA target. Report maximum continuous lateness
Product freshness	Per declared product window, $\geq 99\%$ of conforming source generations publish in objective; 99.9% is GA target
Correctness	Zero unexplained missing keys, duplicate final effects, uncovered CDC intervals, silent coercions, or mismatched generation bindings
Reconciliation	100% of injected violations detected; every exception and declared-scope exclusion reproduces from retained evidence
Exception delivery	100% open within detection deadline, carry complete evidence, route to named owner, and escalate at SLA expiry
Legality	L01 admission reproduces D1–D8 proof; rule-table miss reproduces absent rule, unmet obligations, evidence, and smallest-change counterfactual
Governed context	Every answer carries product generation, freshness, lineage, purpose decision, and masking state; no unauthorized PII release
Recovery	All injected worker and control-plane failures recover inside declared RTO without manual data repair
Authentication	All tested expiry/rotation paths recover without lost checkpoint; revocation blocks new access
Operations	Every late, blocked, failed-test, or overdue-exception state surfaces before customer report; shared causes correlate
Scale	Elastic scale within each partner's declared ceiling; separate synthetic 1→100 test with no contract or commit-semantic change
Connector kit	Independent team reaches Provisional acquisition provider without runtime changes
Unit economics	Fully loaded cost reported by component; $> \$1,000$ per contract-month at cohort volume disproves per-contract pricing before expansion
Service window	30 consecutive production days for the complete Revenue-to-Cash bundle at both partners

Partner acknowledgement, closure, and exception-opening rates are reported cohort observations alongside the calibrated expected envelope; they are not platform exit gates.

A provider-outage exclusion is permitted only when defined before activation and supported by contemporaneous official provider status evidence or independent platform probes showing the fault outside the governed service boundary. Operators cannot apply exclusions retroactively. Every exclusion remains visible in numerator, denominator, duration, and maximum-lateness reporting.

The \$1,000 ceiling is an order-of-magnitude decision threshold, not a price promise. It includes warehouse compute/storage, measured source load, platform compute, staging, evidence retention, and egress. Exceeding it requires a different unit, packaging, or commercial model before provider expansion, supplying cohort evidence for the architecture specification's economic-alignment argument.

15.3 Decision after cohort

Proceed to broader provider certification only if the cohort shows that the coherent outcome is maintainable and that the curated legality mechanism admits and rejects candidates reproducibly. It

does not prove general equivalence, cost optimization, autonomous migration, hysteresis, or optimizer stability. Negative evidence narrows the story before streaming, Kafka, action contracts, or autonomy. The connector-kit gate is independently dispositive: failure to reach Provisional without runtime changes blocks expansion even when the vertical story runs.

15.4 Partner end-state

Before recruitment, each partner receives three end states. If the cohort decision is positive, the contract converts to a time-bounded supported early-production agreement with named service owner and commercial terms. If evaluation continues, operation remains a supported pilot with explicit expiry and no expanded semantics. If the decision is negative or the partner exits, the platform pauses at a checkpoint, exports contract/evidence/checkpoint records, revokes credentials and grants, deletes transient staging on schedule, applies agreed retention to audit evidence, and confirms whether destination data remains partner-owned. Support and incident ownership continue through the agreed wind-down date.

16 Risks and Containment

Table 20. *Principal MVP risks*

Risk	Containment	Stop or redesign trigger
Desktop models produce inconsistent drafts	Server-side typed deltas, bounded tools, deterministic validation, host matrix	Material semantic delta persists after the same authoritative answers
Provider semantics cannot be certified economically	Provisional tier, narrow capabilities, shared harness, measured effort	Reference provider exceeds 8 engineer-months without reusable evidence
Mutable-source replay is not reproducible	Pinned snapshot/LSN/cursor or persisted extraction segment	Provider lacks a stable boundary and reconciliation cannot bound loss
Snowflake commit ambiguity escapes	Stable batch identity, commit query, checkpoint-after-receipt	No provider mechanism can resolve or safely replay commit
Elasticity overloads source	Quota-aware admission, source-load ceiling, conservative scale controller	Freshness requires violating an approved source ceiling
MCP host behavior diverges	Common subset and server-owned state	A required approval or resource cannot be represented safely in one host
Cross-system identity is ambiguous	Require authoritative keys or version-pinned dbt mapping; Unknown blocks Conforming	Pilot needs probabilistic matching or recurring manual identity repair
Declared scope loses usefulness	Count/classify exclusions; named owners; minimum volume, coverage, and drift gates	Scope falls below threshold or unexplained exclusion growth >5 points for two months
Lifecycle deadlines misfit reality	Shadow baseline, separate business/detection clocks, agreed percentile and process obligation	False exceptions exceed real exceptions after two calibration rounds

Risk	Containment	Stop or redesign trigger
External dbt environment is not reproducible	Pin project, manifest, model graph, environment, sources, tests, and receipts	Same inputs and project digest produce materially different product state
Salesforce certification or approval overruns	Start connected-app review in M0; certify narrow objects and semantics first	Exceeds 8 provider-engineer months without reusable conformance evidence
Hiring misses staffing curve	Recruit blocking leads early; eight-week onboarding; gated phase starts	Blocking role is not productive at the phase gate
Agent context leaks governed data	Purpose-bound resources, field policy, redaction, provenance, adversarial prompt tests	Host or model can bypass server-side field and row authorization
Qualified partner pool is too small	Source in week one; evaluation agreements; declared-scope fallback; friendly narrow first bundle	No partner clears screening within 12 weeks; revisit vertical and screen before phase 1
Managed service becomes manual operations	Automated control loops and intervention accounting	Pilot requires recurring per-contract data repair or infrastructure tuning
Kafka omission creates hidden bottleneck	Measure work queue, evidence, and staging limits	Independent fan-out or replay dominates cohort workload

17 Immediate Next Actions

1. Name accountable owners and backups for semantic kernel, runtime/state, providers, desktop authoring, data-product/reconciliation, security, and service operations.
2. Begin partner sourcing against \$15.1 in week one, in parallel with evaluation-agreement and inventory work; revisit the vertical and screen if none clears within 12 weeks.
3. Execute partner inventory: schemas, volumes, process latencies, network, dbt, policy, the unchanged identity screen, and proposed declared reconciliation scope before cohort signature.
4. Build the golden story fixture from at least one partner's actual object/table shapes, key coverage, volume, change rate, and observed process timings; sanitize values but not semantics.
5. Initiate connected-app registration and security review with both partners' Salesforce administrators at project start.
6. Freeze M0 to one PostgreSQL order row, one Snowflake relation, one desktop-authored contract, and one traced live transaction; do not front-load the complete bundle.
7. Write bundle history, activation digest, data-product generation, exception evidence, and legality-decision specifications before implementing the conversational flow.
8. Run Snowflake ambiguous-commit and PostgreSQL snapshot/LSN experiments; promote results into provider declarations.
9. Prove dedicated partner-owned dbt job invocation and actual-source-generation verification without modifying finance SQL.

10. Define provider conformance fixtures, Salesforce semantic cases, lifecycle-rule cases, L01 proof obligations, and one rule-table-miss No Valid Plan case.
11. Instrument bytes, rows, partitions, checkpoints, dbt runs, reconciliation clocks and dispositions, context denials, provider calls, human interventions, and monthly unit economics from M0.
12. Review fresh M0 evidence at six weeks and fund story authoring only if artifact boundaries remain intact.

First funding decision: Approve Phase 0 only. The next decision is based on a fresh PostgreSQL-to-Snowflake transaction whose contract, graph, input boundary, destination receipt, and evidence can be independently traced.

References

- Enterprise Data Compiler, Foundational Architecture Specification v0.3, August 2026.
- Apache Spark. RDD Programming Guide. <https://spark.apache.org/docs/latest/rdd-programming-guide.html>
- Apache Kafka. Design and Delivery Semantics. <https://kafka.apache.org/documentation/#semantics>
- Fivetran. Architecture and deployment documentation. <https://fivetran.com/docs/getting-started/architecture>
- Fivetran. Salesforce connector documentation. <https://fivetran.com/docs/connectors/applications/salesforce>
- Salesforce. Accessing Object Data with Salesforce Platform APIs. <https://developer.salesforce.com/blogs/2024/04/accessing-object-data-with-salesforce-platform-apis>
- Airbyte. Connector catalog and connector development documentation. <https://docs.airbyte.com/integrations/>
- SnapLogic. Designer documentation. <https://docs.snaplogic.com/designer/designer.html>
- Informatica. MDM Customer 360 overview. <https://docs.informatica.com/master-data-management/customer-360/10-5/installation-and-configuration-guide/introduction-to-informatica-mdm---customer-360/overview-of-informatica-mdm---customer-360.html>
- MuleSoft. Mule Runtime Engine overview and API-led connectivity. <https://docs.mulesoft.com/mule-runtime/4.6/>
- Model Context Protocol. Architecture. <https://modelcontextprotocol.io/docs/learn/architecture>
- OpenAI. Codex MCP documentation. <https://developers.openai.com/codex/mcp/>
- Anthropic. Claude MCP documentation. <https://docs.anthropic.com/en/docs/claude-code/mcp>